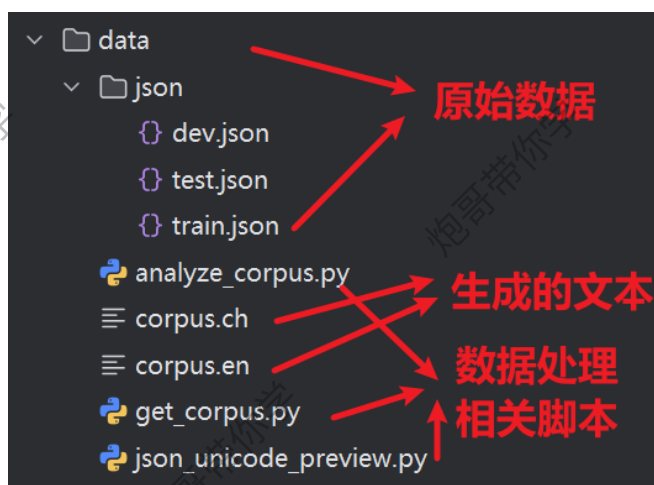


transformer 实战操作文档

一、 数据部分

1.1、数据处理相关代码与数据介绍

该部分是关于代码中原始数据和相关处理部分，具体如下图所示：



(1) 数据来源

实验数据来源“2018 年机器翻译研讨会”，是一个由 统计机器翻译和神经机器翻译领域的研究人员和开发人员组成的会议。在 WMT 2018 的新闻翻译任务中，语言对包括 “Chinese↔English”即中-英 / 英-中方向。[数据网址](#)。

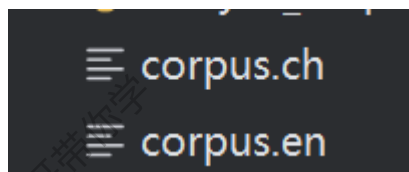
(2) 利用 `json_unicode_preview.py` 脚本，即可查看文本原本样式：

本质上，就是在一个列表中有 一句英文对应一句翻译中文。

```
[
  [
    "One is the recent decision to levy a tax on unused land that could be developed for housing.",
    "其一是不久前做出的对本可用于住宅开发的未利用土地征税的决策。"
  ],
  [
    "What is particularly problematic in the EU is the clash between radical politics and mainstream governance.",
    "欧洲问题的严重之处在于激进政治与主流治理之间的碰撞。"
  ]
]
```

(3) 利用 `get_corpus.py` 脚本，即可对原始的 json 文件进行处理，将对应的中文和英文句子抽离出来，放入到对应的文件中，该操作是方便后续利用对应的

中文和英文文本制作对应的分词器和词表。生成的文件如下：



在担任总理取得实权后，她最终可能会重新审视她与穆沙拉夫的协议。
研究者的结论是，在参加了医疗补助的人中间，自我报告的健康和抑郁情况有所好转，并且这一群体的糖尿病在斯瓦特河谷和巴基斯坦西北部有着大量重要的佛教文物。
但霍华德·休斯作为电影制片人和航空公司老板的成功使得他跻身20世纪前半叶最富有的美国人行列。
他在文章结尾的被他的崇拜者所广泛引用：“我们之所以在构建我们的模型的过程中排除掉所有这些故事，并但担忧绝不仅限于华盛顿，许多美国和其他国家的普罗大众也对特朗普政府的后果真正地忧心忡忡。
诺贝尔经济学奖得主埃德蒙德·菲尔普斯（Edmund S. Phelps）说特朗普直接干预公司部门让人想起了社
这些领导人不太可能接受任何包括塔利班的权力分治。
标准普尔500市盈率正逐渐攀升到16倍的长期均值。

Military leaders know this, and the threat that they will eventually push
The researchers concluded that self-reported overall health and depression
There is a vast number of important Buddhist sites in Swat and other area
But Howard Hughes's success as a film producer and airline owner made him
The concluding sentence of his review is widely quoted by his admirers: "
But the concern goes beyond Washington: many ordinary citizens in the US
The Nobel laureate economist Edmund S. Phelps has described Trump's direc
These leaders are unlikely to accept any power-sharing arrangement that i

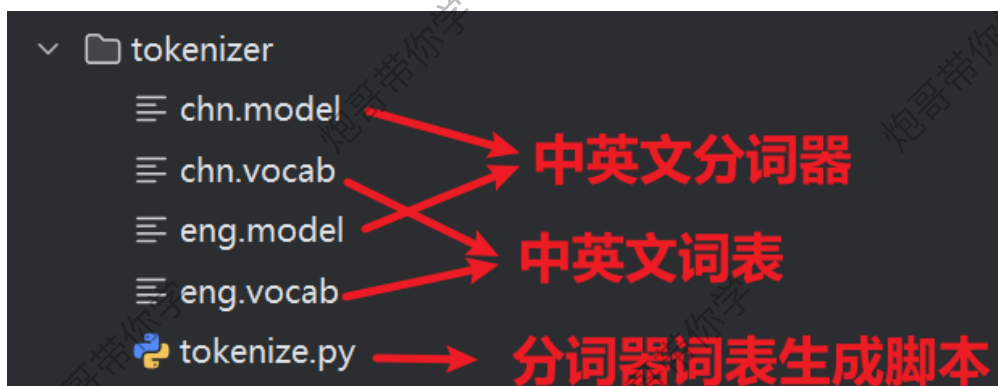
(4) 利用 `analyze_corpus.py` 脚本，即可对的 `corpus.ch`、`corpus.en` 进行分析

```
中文文件 (corpus.ch):  
文件大小: 29.90 MB  
总行数: 252,777  
总字符数: 10,626,053  
平均每行字符数: 42.0  
  
英文文件 (corpus.en):  
文件大小: 34.12 MB  
总行数: 252,777  
总字符数: 34,949,009  
平均每行字符数: 138.3
```

从分析的结果来看，英文句子和中文句子有 252777 行，数据是对应的，这个数据做一个的翻译任务也是可以的。

1.2、数据处理相关代码与数据介绍

相关脚本和生成的中英文分词器词表，具体如下图所示：



.model 是“分词器的大脑”——一个二进制模型，包含如何把字符串切成子词、如何把子词变回文本，以及所有规则与 ID 映射。

```
ENQ<unk>NAKNULNULNULCANSTX
FF
ETX<s>NAKNULNULNULCANETX
EOT</s>NAKNULNULNULCANETX
ACK—NAKNULNULNUL
ACK经济NAKNULNUL
ACK国家NAKNULNUL
```

.vocab 是“人可读的清单”——一个纯文本表，按行列出每个子词和它的分数（常是对数概率或权重），行号即 token 的 ID 顺序。

—	-0	_t	-0
经济	-1	in	-1
国家	-2	_a	-2
美国	-3	he	-3
_但	-4	re	-4
一个	-5	on	-5

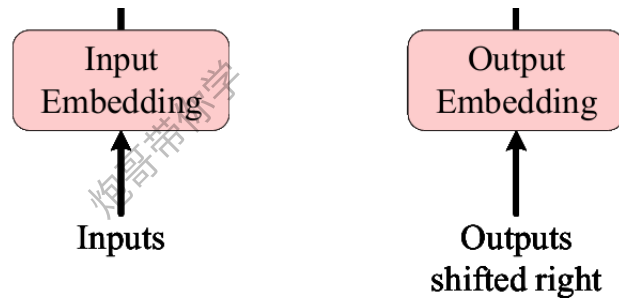
二、代码模型结构部分

模型结构损失函数相关代码，具体如下图所示：



(1) 词嵌入层(Token Embedding)

Transformer 的“词嵌入层”，属于输入表示模块，通常与“位置编码”并列为输入端的一部分（Encoder 和 Decoder 都会用到）。



```
class Embeddings(nn.Module):
    def __init__(self, d_model, vocab):
        # 初始化方法，传入模型的维度（d_model）和词汇表的大小（vocab）
        super(Embeddings, self).__init__()
        # Embedding层，将词汇表的大小映射为d_model维的向量
        self.lut = nn.Embedding(vocab, d_model)
        # 存储模型的维度 d_model
        self.d_model = d_model

    def forward(self, x):
        # 返回x对应的embedding矩阵（需要乘以math.sqrt(d_model)）
        # 这是为了保持词向量的方差，使其适应后续层的训练。
        return self.lut(x) * math.sqrt(self.d_model)
```

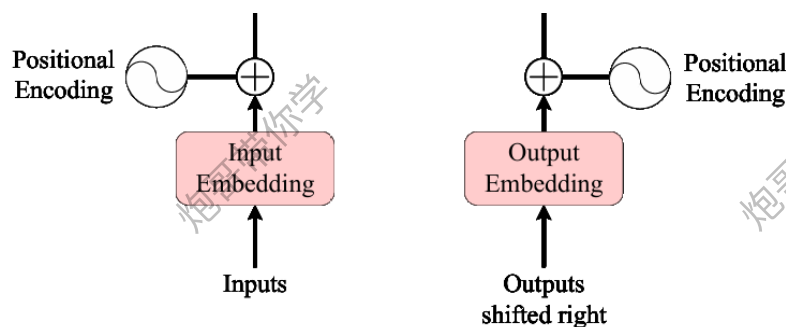
输出乘以 $\sqrt{d_model}$ ：这是 Transformer 论文里的做法，使嵌入向量的量级与后续残差/位置编码在同一尺度，稳定训练、加快收敛。

(2) 位置编码层

位置编码的相关公式

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right) \quad PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{\frac{2i}{d}}}\right)$$

这段代码实现了 位置编码 (Positional Encoding)，它通过使用正弦和余弦函数对每个位置进行编码，并将其与输入的词嵌入向量相加，从而提供序列中单词之间的顺序信息。



```
class PositionalEncoding(nn.Module): 2 usages
    def __init__(self, d_model, dropout, max_len=5000):
        super(PositionalEncoding, self).__init__()
        self.dropout = nn.Dropout(p=dropout)

        # 初始化一个大小为 max_len(设定的最大长度)*embedding维度 的全零矩阵
        # 来存放所有小于这个长度位置对应的positional embedding
        pe = torch.zeros(max_len, d_model, device=DEVICE)
        # 生成一个位置下标的tensor矩阵(每一行都是一个位置下标)
        position = torch.arange(0., max_len, device=DEVICE).unsqueeze(1)
        # 这里幂运算太多，我们使用exp和log来转换实现公式中pos下面要除以的分母（由于是分母，要注意带负号）
        div_term = torch.exp(torch.arange(0., d_model, 2, device=DEVICE) * -(math.log(10000.0) / d_model))

        # 根据公式，计算各个位置在各embedding维度上的位置纹理值，存放到pe矩阵中
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)

        # 加1个维度，使得pe维度变为：1*max_len*embedding维度
        # （方便后续与一个batch的句子所有词的embedding批量相加）
        pe = pe.unsqueeze(0)
        # 将pe矩阵以持久的buffer状态存下（不会作为要训练的参数）
        self.register_buffer(name='pe', pe)

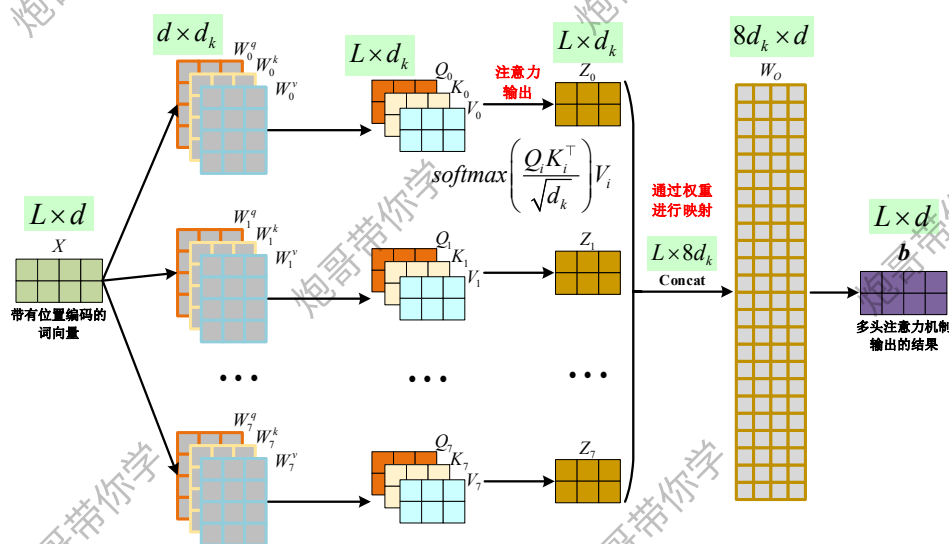
    def forward(self, x):
        # 将一个batch的句子所有词的embedding与已构建好的positional embedding相加
        # （这里按照该批次数据的最大句子长度来取对应需要的那些positional embedding值）
        x = x + Variable(self.pe[:, :x.size(1)], requires_grad=False)
        return self.dropout(x)
```

(3) 注意力机制函数与多头注意力机制网络

attention 函数实现注意力计算的核心部分。计算 query 和 key 之间的点积，并根据该相似度对 value 进行加权求和。

MultiHeadedAttention 实现了多头自注意力机制。该机制通过将输入的 query、key 和 value 分配给多个头，每个头独立计算注意力权重，然后将各个头的输出拼接在一起，最后通过一个线性变换得到最终结果。

具体网络模型结构图和对应的代码如图所示：



```
def attention(query, key, value, mask=None, dropout=None):
    # 将query矩阵的最后一个维度值作为d_k
    d_k = query.size(-1)

    # 将key的最后两个维度互换(转置)，才能与query矩阵相乘，乘完了还要除以d_k开根号
    scores = torch.matmul(query, key.transpose(-2, -1)) / math.sqrt(d_k)

    # 如果存在要进行mask的内容，则将那些为0的部分替换成一个很大的负数
    if mask is not None:
        scores = scores.masked_fill(mask == 0, -1e9)

    # 将mask后的attention矩阵按照最后一个维度进行softmax
    p_attn = F.softmax(scores, dim=-1)

    # 如果dropout参数设置为非空，则进行dropout操作
    if dropout is not None:
        p_attn = dropout(p_attn)

    # 最后返回注意力矩阵跟value的乘积，以及注意力矩阵
    return torch.matmul(p_attn, value), p_attn
```

```

class MultiHeadedAttention(nn.Module): 2 usages
    def __init__(self, h, d_model, dropout=0.1):
        super(MultiHeadedAttention, self).__init__()
        # 保证可以整除
        assert d_model % h == 0
        # 得到一个head的attention表示维度
        self.d_k = d_model // h
        # head数量
        self.h = h
        # 定义4个全连接函数，供后续作为WQ, WK, WV矩阵和最后h个多头注意力矩阵concat之后进行变换的矩阵
        self.linears = clones(nn.Linear(d_model, d_model), N: 4)
        self.attn = None
        self.dropout = nn.Dropout(p=dropout)

    def forward(self, query, key, value, mask=None):
        if mask is not None:
            mask = mask.unsqueeze(1)
        # query的第一个维度值为batch size
        nbatches = query.size(0)
        # 将embedding层乘以WQ, WK, WV矩阵(均为全连接)
        # 并将结果拆成h块，然后将第二个和第三个维度值互换(具体过程见上述解析)
        query, key, value = [l(x).view(nbatches, -1, self.h, self.d_k).transpose(1, 2)
                               for l, x in zip(self.linears, (query, key, value))]
        # 调用上述定义的attention函数计算得到h个注意力矩阵跟value的乘积，以及注意力矩阵
        x, self.attn = attention(query, key, value, mask=mask, dropout=self.dropout)
        # 将h个多头注意力矩阵concat起来(注意要先把h变回到第三维的位置)
        x = x.transpose(dim0: 1, dim1: 2).contiguous().view(nbatches, -1, self.h * self.d_k)
        # 使用self.linears中构造的最后一个全连接函数来存放变换后的矩阵进行返回
        return self.linears[-1](x)

```

(4) 层归一化

$$LayerNorm(x) = \frac{x - \mu}{\sigma} \cdot \gamma + \beta$$

层归一化公式：

```

class LayerNorm(nn.Module): 4 usages
    def __init__(self, features, eps=1e-6):
        super(LayerNorm, self).__init__()
        # 初始化α为全1，而β为全0
        self.a_2 = nn.Parameter(torch.ones(features))
        self.b_2 = nn.Parameter(torch.zeros(features))
        # 平滑项
        self.eps = eps

    def forward(self, x):
        # 这是一个前向传播函数，用于执行Layer Normalization操作
        # 输入x是神经网络层的输出
        # 按最后一个维度计算均值和方差
        # keepdim=True确保输出的维度与输入相同
        mean = x.mean(-1, keepdim=True) # 计算最后一个维度的均值
        std = x.std(-1, keepdim=True) # 计算最后一个维度的标准差

        # 返回Layer Norm的结果
        # Layer Norm公式: y = a * (x - mean) / sqrt(std^2 + eps) + b
        # 其中a和b是可学习的参数，eps是为了防止除以0的小常数
        return self.a_2 * (x - mean) / torch.sqrt(std ** 2 + self.eps) + self.b_2

```

(5) 前馈神经网络

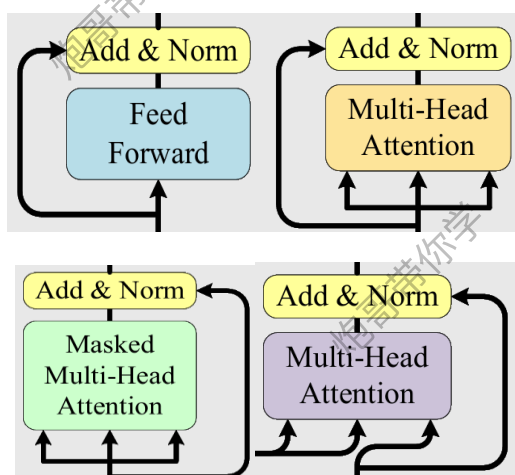
PositionwiseFeedForward: 是 Transformer 模型中的前馈神经网络。它包含两个线性层和一个 ReLU 激活函数，通过 dropout 防止过拟合。

```
class PositionwiseFeedForward(nn.Module): 2 usages
    def __init__(self, d_model, d_ff, dropout=0.1):
        """
        位置前馈神经网络初始化函数
        参数:
            d_model: 模型的输入维度
            d_ff: 前馈神经网络中间层的维度
            dropout: dropout 概率, 默认为0.1
        """
        super(PositionwiseFeedForward, self).__init__()
        self.w_1 = nn.Linear(d_model, d_ff) # 第一个线性层,
        self.w_2 = nn.Linear(d_ff, d_model) # 第二个线性层,
        self.dropout = nn.Dropout(dropout) # dropout层,

    def forward(self, x):
        return self.w_2(self.dropout(F.relu(self.w_1(x))))
```

(6) SublayerConnection 函数部分

SublayerConnection 是 Transformer 模型中的一个关键组件，它负责将各个子层（如多头注意力层、前馈神经网络层）与残差连接和层归一化相结合。每个 Transformer 层的输出都经过该处理，以确保信息在模型中能够有效流动，并防止梯度消失问题。



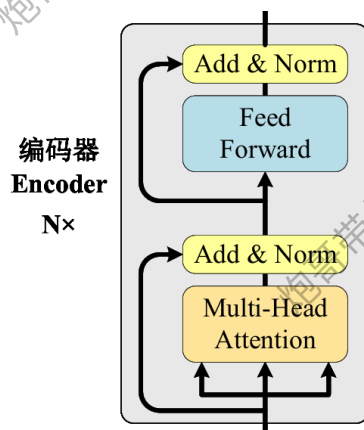
```
class SubLayerConnection(nn.Module):
    """
    SubLayerConnection的作用就是把Multi-Head Attention和Feed Forward层
    连在一起只不过每一层输出之后都要先做Layer Norm再残差连接
    sublayer是lambda函数
    """
    def __init__(self, size, dropout):
        super(SubLayerConnection, self).__init__()
        self.norm = LayerNorm(size)
        self.dropout = nn.Dropout(dropout)

    """
    sublayer参数是SubLayerConnection类中的核心组件，
    它代表了Transformer中的具体处理层，通过这种设计实现了代码的模块化和灵活性。
    """
    def forward(self, x, sublayer):
        # 返回Layer Norm和残差连接后结果
        return x + self.dropout(sublayer(self.norm(x)))
```

(7) 编码器层与编码器部分

EncoderLayer 是 Transformer 模型中的核心组件之一，它实现了单个编码器层的结构。每个编码器层由两个主要子层组成：多头自注意力层和前馈神经网络，并且包含残差连接和层归一化。

Encoder 由多个 EncoderLayer 堆叠而成，它是 Transformer 模型的编码器部分。Encoder 将输入的序列经过多层编码层 (EncoderLayer)，从而学习输入序列的上下文信息。



```

class EncoderLayer(nn.Module):
    def __init__(self, size, self_attn, feed_forward, dropout):
        super(EncoderLayer, self).__init__()
        self.self_attn = self_attn
        self.feed_forward = feed_forward
        # SublayerConnection的作用就是把multi和ffn连在一起
        # 只不过每一层输出之后都要先做Layer Norm再残差连接
        self.sublayer = clones(SublayerConnection(size, dropout), N=2)
        # d_model
        self.size = size

    def forward(self, x, mask):
        # 将embedding层进行Multi head Attention
        x = self.sublayer[0](x, lambda x: self.self_attn(x, x, x, mask))
        # 注意到attn得到的结果x直接作为下一层的输入
        return self.sublayer[1](x, self.feed_forward)

```

```

class Encoder(nn.Module):
    # layer = EncoderLayer
    # N = 6
    def __init__(self, layer, N):
        super(Encoder, self).__init__()
        # 复制N个encoder layer
        self.layers = clones(layer, N)
        # Layer Norm
        self.norm = LayerNorm(layer.size)

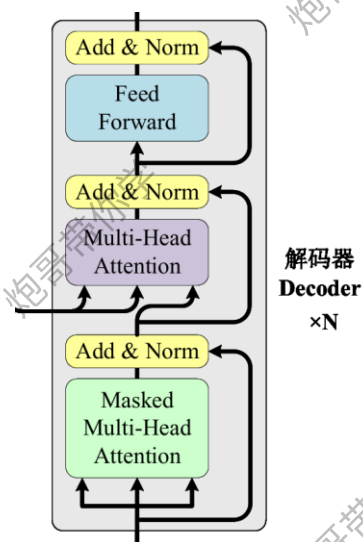
    def forward(self, x, mask):
        """
        使用循环连续encode N次(这里为6次)
        这里的Encoderlayer会接收一个对于输入的attention mask处理
        """
        for layer in self.layers:
            x = layer(x, mask)
        return self.norm(x)

```

(8) 解码器层与解码器部分

DecoderLayer 是 Transformer 模型中的核心组件之一，负责解码器层的计算。每个解码器层包含自注意力机制、与编码器输出的上下文进行的注意力机制以及前馈神经网络。这些子层通过残差连接和层归一化相连。

Decoder 是 Transformer 模型的解码器部分，由多个 DecoderLayer 堆叠而成。它将输入的上下文（来自编码器的输出 memory）和目标序列 x 逐层传递，生成最终的输出。



```
class DecoderLayer(nn.Module):
    def __init__(self, size, self_attn, src_attn, feed_forward, dropout):
        super(DecoderLayer, self).__init__()
        self.size = size
        # Self-Attention
        self.self_attn = self_attn
        # 与Encoder传入的Context进行Attention
        self.src_attn = src_attn
        self.feed_forward = feed_forward
        self.sublayer = clones(SublayerConnection(size, dropout), N:3)

    def forward(self, x, memory, src_mask, tgt_mask):
        # 用m来存放encoder的最终hidden表示结果
        m = memory

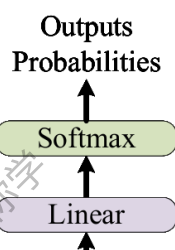
        # Self-Attention: 注意self-attention的q, k和v均为decoder hidden
        x = self.sublayer[0](x, lambda x: self.self_attn(x, x, x, tgt_mask))
        # Context-Attention: 注意context-attention的q为decoder hidden, 而k和v为encoder hidden
        x = self.sublayer[1](x, lambda x: self.src_attn(x, m, m, src_mask))
        return self.sublayer[2](x, self.feed_forward)
```

```
class Decoder(nn.Module):
    def __init__(self, layer, N):
        super(Decoder, self).__init__()
        # 复制N个encoder layer
        self.layers = clones(layer, N)
        # Layer Norm
        self.norm = LayerNorm(layer.size)

    def forward(self, x, memory, src_mask, tgt_mask):
        """
        使用循环连续decode N次(这里为6次)
        这里的DecoderLayer会接收一个对于输入的attention mask处理
        和一个对输出的attention mask + subsequent mask处理
        """
        for layer in self.layers:
            x = layer(x, memory, src_mask, tgt_mask)
        return self.norm(x)
```

(9) 额外函数部分

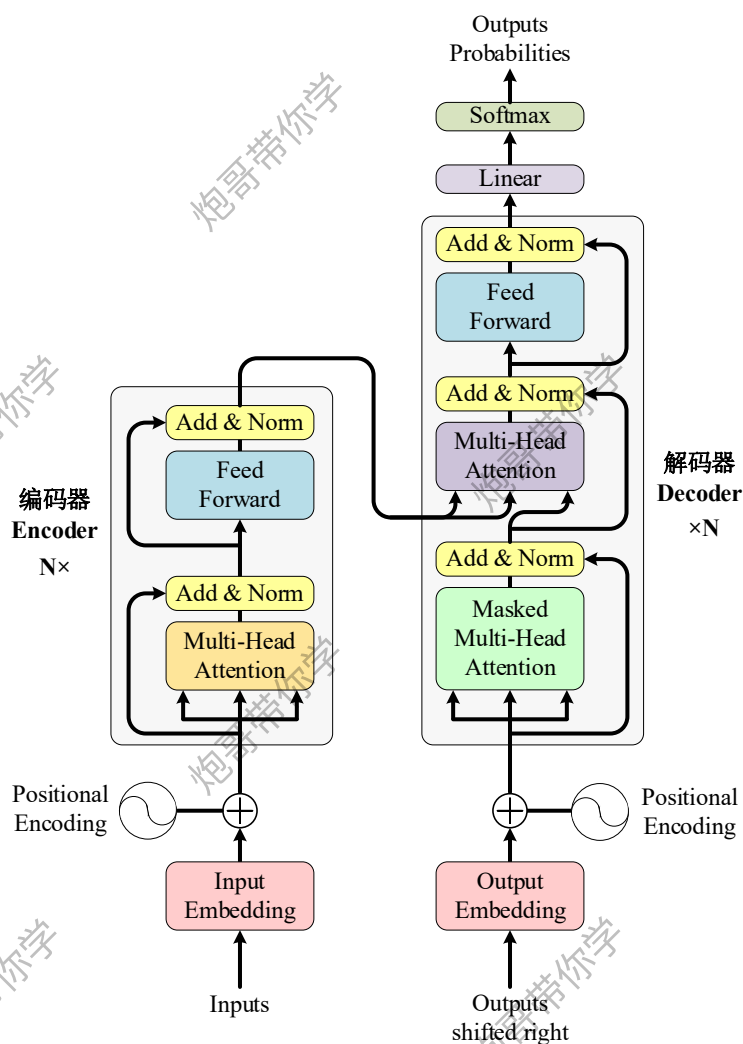
Generator 是 Transformer 模型中的生成器部分，主要负责将解码器的输出转换为 目标词汇表的概率分布。



```
class Generator(nn.Module):
    # vocab: tgt_vocab
    def __init__(self, d_model, vocab):
        super(Generator, self).__init__()
        # decode后的结果, 先进入一个全连接层变为词典大小的向量
        self.proj = nn.Linear(d_model, vocab)

    def forward(self, x):
        # 然后再进行log_softmax操作(在softmax结果上再做一次log运算)
        return F.log_softmax(self.proj(x), dim=-1)
```

(10) Transformer 整体结构定义



```
class Transformer(nn.Module):
    def __init__(self, encoder, decoder, src_embed, tgt_embed, generator):
        super(Transformer, self).__init__()
        self.encoder = encoder
        self.decoder = decoder
        self.src_embed = src_embed
        self.tgt_embed = tgt_embed
        self.generator = generator

    2 usages (1 dynamic)
    def encode(self, src, src_mask):
        return self.encoder(self.src_embed(src), src_mask)

    2 usages (1 dynamic)
    def decode(self, memory, src_mask, tgt, tgt_mask):
        return self.decoder(self.tgt_embed(tgt), memory, src_mask, tgt_mask)

    def forward(self, src, tgt, src_mask, tgt_mask):
        # encoder的结果作为decoder的memory参数传入, 进行decode
        return self.decode(self.encode(src, src_mask), src_mask, tgt, tgt_mask)
```

```
def make_model(src_vocab, tgt_vocab, N=6, d_model=512, d_ff=2048, h=8, dropout=0.1):
    c = copy.deepcopy
    # 实例化Attention对象
    attn = MultiHeadedAttention(h, d_model).to(DEVICE)
    # 实例化FeedForward对象
    ff = PositionwiseFeedForward(d_model, d_ff, dropout).to(DEVICE)
    # 实例化PositionalEncoding对象
    position = PositionalEncoding(d_model, dropout).to(DEVICE)
    # 实例化Transformer模型对象

    model = Transformer(
        Encoder(EncoderLayer(d_model, c(attn), c(ff), dropout).to(DEVICE), N).to(DEVICE),
        Decoder(DecoderLayer(d_model, c(attn), c(attn), c(ff), dropout).to(DEVICE), N).to(DEVICE),
        nn.Sequential(Embeddings(d_model, src_vocab).to(DEVICE), c(position)),
        nn.Sequential(Embeddings(d_model, tgt_vocab).to(DEVICE), c(position)),
        Generator(d_model, tgt_vocab)).to(DEVICE)

    # 初始化模型参数
    # 遍历模型中的所有参数
    for p in model.parameters():
        # 判断参数是否为二维或更高维（例如权重矩阵，而不是偏置向量）
        if p.dim() > 1:
            # 这里初始化采用的是nn.init.xavier_uniform
            nn.init.xavier_uniform_(p)
    return model.to(DEVICE)
```

三、 模型训练推理部分

```
🐍 beam_decoder.py
🐍 config.py 训练推理参数脚本
🐍 main.py 模型训练脚本
📄 requirements.txt
🐍 translate.py 模型推理脚本
📄 版本.txt
```

(1)、模型训练

可调训练参数如下所示：

```
"""
训练配置
这些参数控制训练过程中的配置和训练策略。
"""

# 训练时的批次大小。
batch_size = 32
# 训练的总轮次。
epoch_num = 40
# 学习率 (learning rate) 。
lr = 3e-4
```

其中 batch 为 32 的时候，占用显存为 12GB，各位同学可以参考自己的显卡去设置对应的 batch 大小。

训练轮次：模型在大概 25-30 轮左右大约收敛（但是不同机器可能收敛的轮次不同）在 5070 显卡上训练，大概 1 轮 1 个小时左右。

学习率可以不调整。

训练数据集路径和权重路径：

```
"""
文件路径和模型配置
这些参数用于定义文件路径和是否加载预训练模型的设置。
"""
data_dir = './data'
train_data_path = './data/json/train.json'
dev_data_path = './data/json/dev.json'
test_data_path = './data/json/test.json'
model_path = './weights/with_NoamOpt_no_LabelSmoothing_model.pth'
test_model_path = './run/train/exp2/weights/best_bleu_26.38.pth'
```

数据集路径

预训练权重

训练结果：

```
2025-11-04 07:24:48,738-root-INF0-第17轮模型训练与验证
100%|██████████| 5530/5530 [07:57<00:00, 11.58it/s]
100%|██████████| 790/790 [00:19<00:00, 40.45it/s]
100%|██████████| 790/790 [18:27<00:00, 1.40s/it]
2025-11-04 07:51:39,507-root-INF0-Epoch: 17, train_loss: 2.832, val_loss: 3.943, Bleu Score: 26.14

2025-11-04 07:51:40,045-root-INF0-第18轮模型训练与验证
100%|██████████| 5530/5530 [12:01<00:00, 7.66it/s]
100%|██████████| 790/790 [00:38<00:00, 20.70it/s]
100%|██████████| 790/790 [35:35<00:00, 2.70s/it]
2025-11-04 08:40:02,801-root-INF0-Epoch: 18, train_loss: 2.777, val_loss: 3.958, Bleu Score: 26.05

2025-11-04 08:40:02,801-root-INF0-第19轮模型训练与验证
100%|██████████| 5530/5530 [08:57<00:00, 10.30it/s]
100%|██████████| 790/790 [00:19<00:00, 40.71it/s]
100%|██████████| 790/790 [19:02<00:00, 1.45s/it]
2025-11-04 09:08:28,272-root-INF0-Epoch: 19, train_loss: 2.726, val_loss: 3.962, Bleu Score: 26.30
```

关于评价指标：

BLEU (Bilingual Evaluation Understudy) **是用于自动评估机器翻译质量的一个常见指标，其评价的是机器翻译输出与人工翻译参考之间的相似度。BLEU 的评分范围从 0 到 100：

0-20: 较差的翻译：翻译结果几乎与参考翻译完全不匹配。

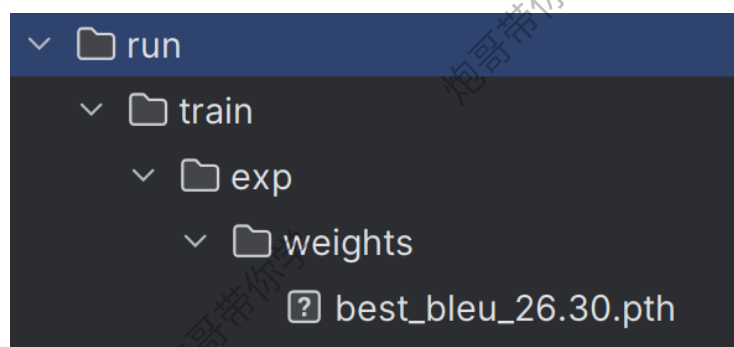
20-40: 一般的翻译：尽管可以理解，但翻译中通常包含错误。

40-60: 中等质量的翻译：翻译与参考翻译有一定程度的一致性。

60-80: 翻译的质量较高，通常能够准确表达参考翻译的内容。

80-100: 极好的翻译，翻译几乎与参考翻译一致。语言流畅且正确。

训练完成以后在 run 文件夹下生成对应的权重文件。



(2)、模型推理

调用推理脚本进行推理，首先设置对应的推理权重路径

```
"""
data_dir = './data'
train_data_path = './data/json/train.json'
dev_data_path = './data/json/dev.json'
test_data_path = './data/json/test.json'
model_path = './weights/with_NoamOpt_no_LabelSmoothing_model.pth'
test_model_path = './run/train/exp2/weights/best_bleu_26.38.pth'
"""
```

模型推理路径

进行机器翻译

```
D:\APP\anaconda\envs\tf1\python.exe C:\Users\Administrator\Desktop\ChineseNMT4\translate.py
请输入英文句子进行翻译: I love you
翻译结果: 我喜欢你
请输入英文句子进行翻译: I am a dog
翻译结果: 我是个狗。
请输入英文句子进行翻译: "The government has implemented various policies to improve the living standards of its citizens."
翻译结果: "政府实施了诸多政策,改善公民的生活水平。"
请输入英文句子进行翻译: |
```